1. Aim and Learning Objectives

The aim of this practical is to understand the basic communication model of ROS 2 and implement a simple drone telemetry system. A publisher generates simulated altitude data; a subscriber can receive and display it in real time. The publisher is then extended to send battery level along with altitude.

Learning objectives

- Understand what a ROS 2 node is and how nodes are organised into a robotic application.
- Understand topics as named communication channels.
- Create and run a publisher node that sends simulated drone altitude.
- Create an optional subscriber node that receives and displays altitude.
- Extend the system to publish altitude and battery percentage together using a custom message.
- Build, source and verify a ROS 2 Python package in ConstructSim.

Practical outcome

The completed system publishes a message once per second. Altitude begins at 100.0 m and increases by 0.5 m each cycle. Battery begins at 100% and decreases by 1 percentage point per cycle. These are simulated values, not readings from a physical drone.

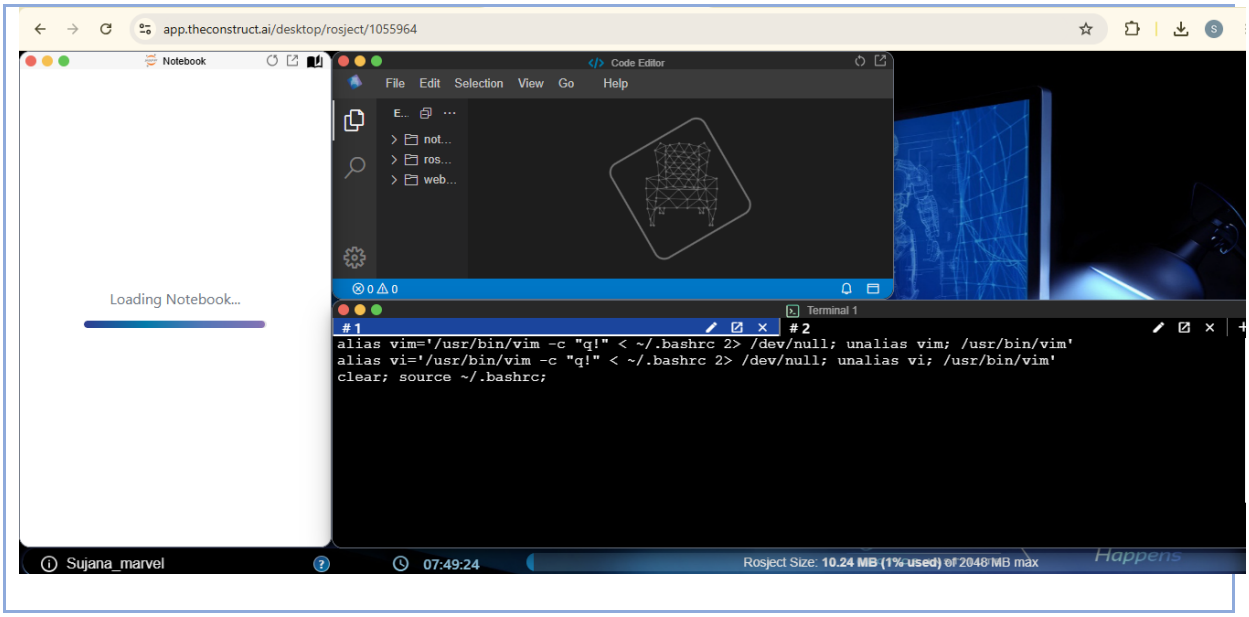


Figure 1 : ROS2 Humble Workspace Using Construct Sim

2. Introduction to ROS 2

ROS 2 (Robot Operating System 2) is a software framework and collection of tools used to build robot applications. Despite the name, it is not a traditional operating system. It provides communication, reusable libraries, package management conventions and command-line tools that help separate robot functions into smaller components.

For example, a real drone application might contain separate nodes for GPS, altitude sensing, battery monitoring, navigation and display. Each node can perform one task and exchange information with other nodes through ROS 2 communication mechanisms.

Why ROS 2 is useful

- Modularity: different robot functions can be implemented as separate nodes.
- Communication: nodes can exchange data without being combined into one large program.
- Reuse: packages and message definitions can be reused across projects.
- Testing and debugging: command-line tools help inspect nodes, topics and messages.
- Scalability: the same concepts can be used for small simulations and larger robotic systems.

3. Core Concepts: Nodes, Topics, Publishers and Subscribers

3.1 Node

A node is a program or process that performs a particular task in a ROS 2 system. In this practical, drone_publisher generates telemetry and drone_subscriber receives it.

3.2 Topic

A topic is a named channel used to send messages. The altitude-only version uses /drone_altitude. The combined altitude-and-battery version uses /drone_data. Topic names should be consistent between the publisher and subscriber.

3.3 Publisher

A publisher sends messages to a topic. It creates a publisher with a message type and topic name, then publishes a message whenever its timer callback runs.

3.4 Subscriber

A subscriber listens to a topic. When a message arrives, ROS 2 calls the subscriber's callback function, which can read the fields and print them to the terminal.

3.5 Message and callback

A message defines the structure of the information being exchanged. A callback is a function that ROS 2 runs in response to an event, such as a timer firing or a message arriving.

3.6 Simple communication flow

Publisher node → Topic → Subscriber node

For the final version: drone_publisher → /drone_data → drone_subscriber. The custom DroneData message contains altitude and battery fields.

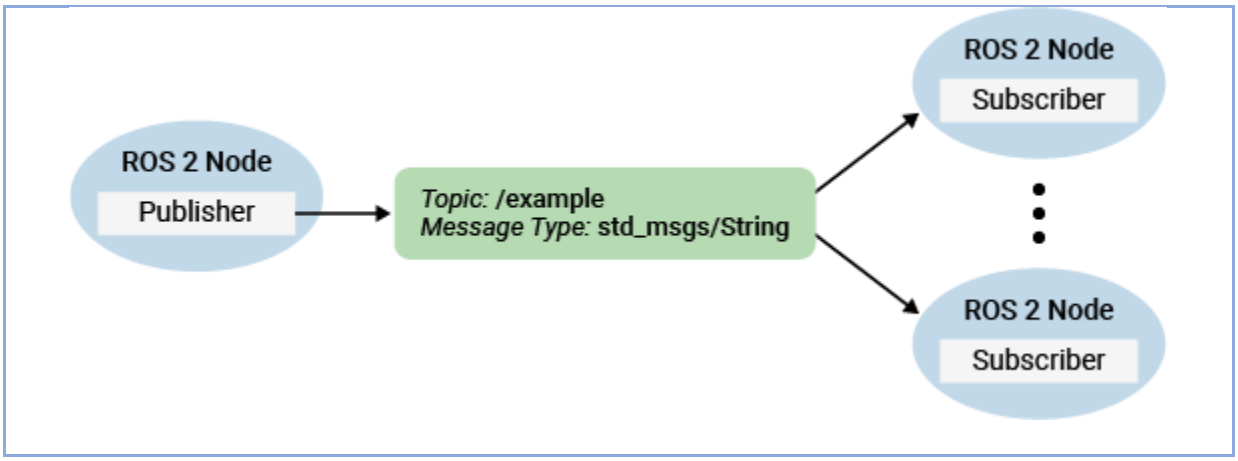


Figure 2 : Basic Block Daigram of ROS System

4. Environment and Workspace Setup

This practical was completed in ConstructSim using ROS 2 Humble and a Python package. The workspace used in the steps below is ~/ros2_ws. If your environment uses a different workspace name, replace the path accordingly.

4.1 Workspace structure

```
~/ros2_ws/
└── src/
    ├── drone_telemetry/
    │   ├── package.xml
    │   ├── setup.py
    │   └── drone_telemetry/
```

```
│       ├── __init__.py
│       ├── drone_publisher.py
│       └── drone_subscriber.py
└── drone_interfaces/
    ├── package.xml
    ├── CMakeLists.txt
    └── msg/
        └── DroneData.msg
```

The drone_telemetry package contains the Python nodes. The drone_interfaces package defines the custom message used to send altitude and battery in one message.

4.2 Create or open the workspace

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

If the packages have already been created in ConstructSim, do not create duplicate packages. Open the existing files in the file explorer and verify their contents.

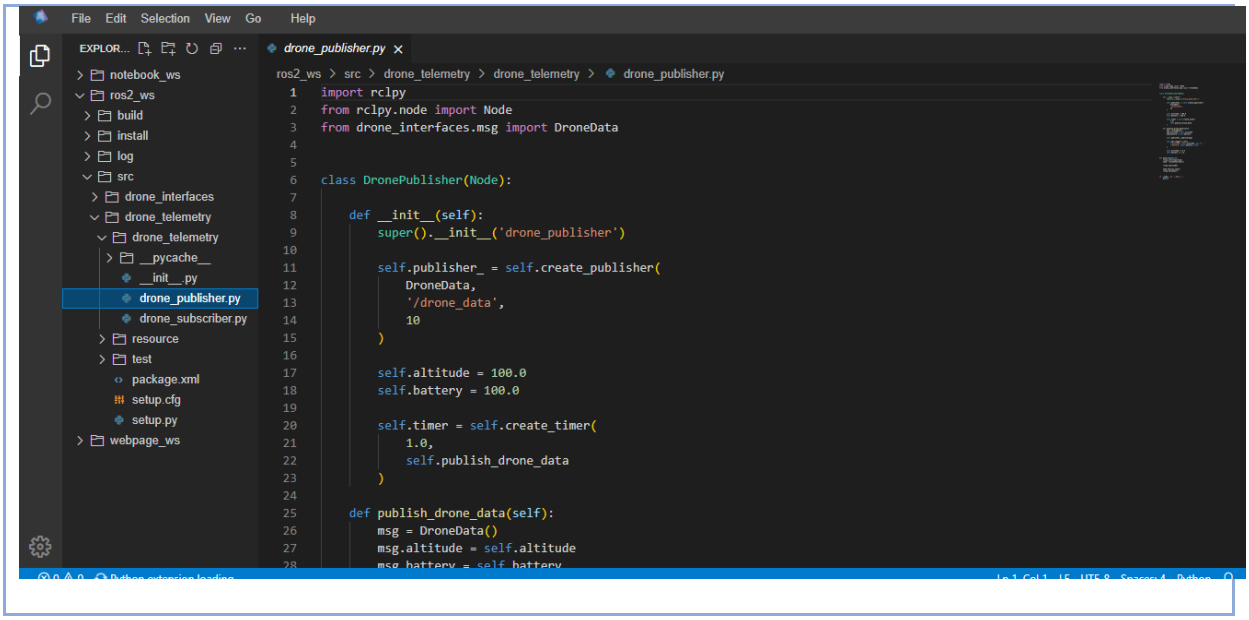


Figure 3: File Explorer of ROS2 Humble

5. Create the Altitude Publisher

The first version publishes a single floating-point altitude value to /drone_altitude once per second. This is the basic publisher stage before adding battery data.

5.1 Publisher code — altitude only

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
class DronePublisher(Node):
def __init__(self):
super().__init__('drone_publisher')
self.publisher_ = self.create_publisher(
        Float32, '/drone_altitude', 10
    )
    self.altitude = 100.0
    self.timer = self.create_timer(1.0, self.publish_altitude)
 def publish_altitude(self):
    msg = Float32()
    msg.data = self.altitude
    self.publisher_.publish(msg)

    self.get_logger().info(
      f'Altitude: {self.altitude:.2f} m'
    )
    self.altitude += 0.5
```

```
def main(args=None):
    rclpy.init(args=args)
    node = DronePublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

5.2 Explanation

- rclpy provides the Python client library for ROS 2.
- Node is the base class used to create a ROS 2 node.
- Float32 is a standard message containing one 32-bit floating-point field named data.
- create_publisher(Float32, '/drone_altitude', 10) creates a publisher on the named topic; 10 is the queue depth used for the QoS history setting.
- create_timer(1.0, ...) calls the publish function approximately every second.
- The altitude starts at 100.0 and increases by 0.5 after each published message.
- rclpy.spin(node) keeps the node running so it can process timers and callbacks.

5.3 Register the executable

In setup.py, the console_scripts entry point should include the publisher. Preserve any existing required setup configuration and ensure this entry is inside entry_points.

```
entry_points={
    'console_scripts': [
        'drone_publisher = drone_telemetry.drone_publisher:main',
    ],
},
```

If the subscriber entry already exists, keep it too. The next section adds that entry.

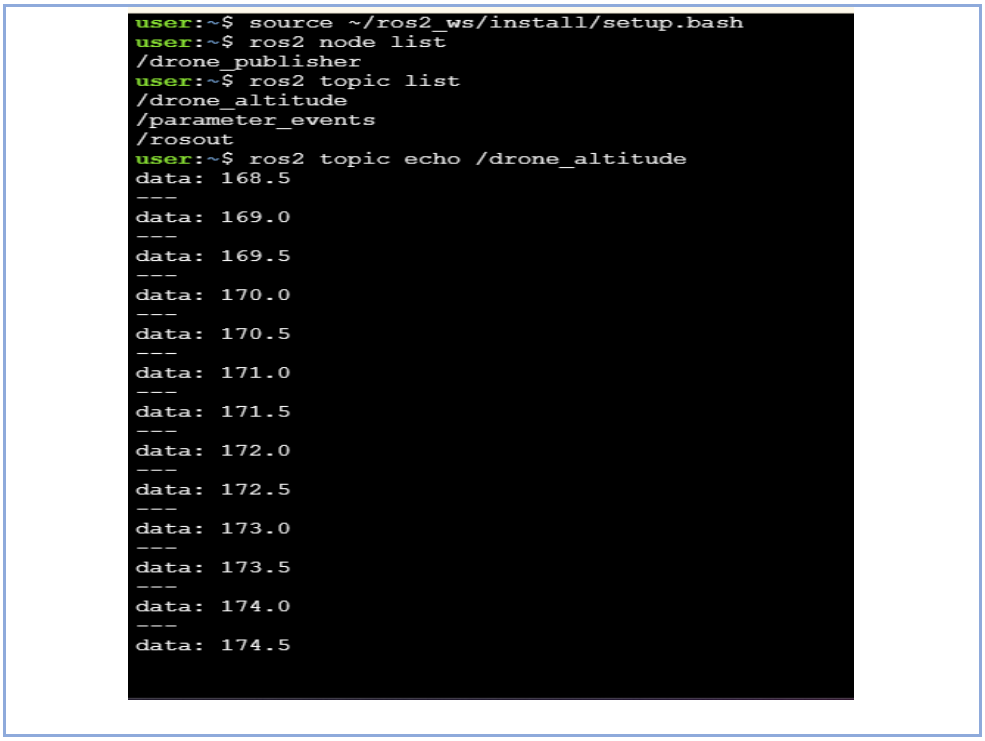


Figure 4: Created a Fake Data using Publisher Node

6. Create the Altitude Subscriber (Optional)

The subscriber listens to /drone_altitude and prints each received value. It is optional in the task, but it is useful for verifying that the publisher's messages are being received.

6.1 Subscriber code — altitude only

```
import rclpy

from rclpy.node import Node

from std_msgs.msg import Float32

class DroneSubscriber(Node):

def __init__(self):

super().__init__('drone_subscriber')

self.subscription = self.create_subscription(

Float32,

'/drone_altitude',
```

```
self.altitude_callback,
            10
        )
    def altitude_callback(self, msg):
        self.get_logger().info(
            f'Received altitude: {msg.data:.2f} m'
        )
    def main(args=None):
      rclpy.init(args=args)
      node = DroneSubscriber()
      rclpy.spin(node)
      node.destroy_node()
      rclpy.shutdown()
    if __name__ == '__main__':
      main()
```

6.2 Register the subscriber

```
  entry_points={
      'console_scripts': [
          'drone_publisher = drone_telemetry.drone_publisher:main',
          'drone_subscriber = drone_telemetry.drone_subscriber:main',
      ],
  },
```

The subscriber's topic name and message type must match the publisher's. Both nodes should be running in separate terminals for live communication.

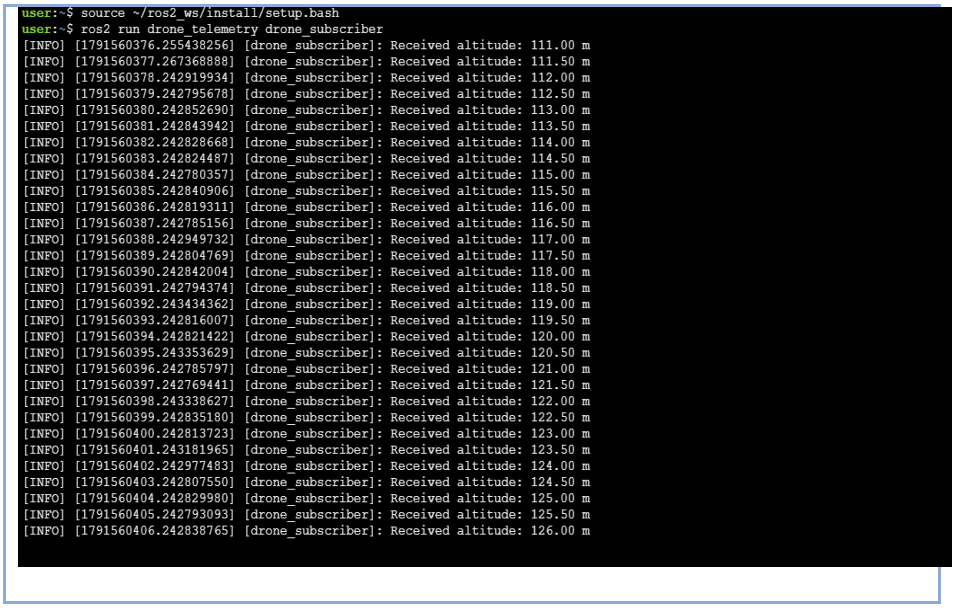


Figure 5: Created a Actual Data using Subscriber Node

7. Extend Telemetry to Include Battery Level

To publish altitude and battery together, the example uses a custom ROS 2 message named DroneData. This avoids sending two unrelated messages when both values belong to the same telemetry update.

7.1 Define the custom message

Create drone_interfaces/msg/DroneData.msg with the following two fields:

```
float32 altitude
float32 battery
```

Each line defines a field type and name. Both values are floating-point numbers. Battery is represented as a percentage from 0 to 100 in this simulation.

7.2 Configure drone_interfaces/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>drone_interfaces</name>
  <version>0.0.0</version>
  <description>Custom messages for drone telemetry</description>
```

```
  <maintainer email="user@example.com">user</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <build_depend>rosidl_default_generators</build_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

7.3 Configure drone_interfaces/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(drone_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/DroneData.msg"
)
 ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

The interface package uses rosidl to generate Python and other language bindings for the custom message. The package name in package.xml and project(...) must match.

7.4 Declare the dependency

In drone_telemetry/package.xml, add the dependency below inside the package element, alongside the other dependencies:

```
<depend>drone_interfaces</depend>
```

Do not add drone_interfaces as a dependency inside drone_interfaces/package.xml itself. A package must not declare a dependency on itself.

7.5 Updated publisher — altitude and battery

```
import rclpy
from rclpy.node import Node
from drone_interfaces.msg import DroneData
class DronePublisher(Node):
def __init__(self):
super().__init__('drone_publisher')
self.publisher_ = self.create_publisher(
DroneData, '/drone_data', 10
        )
        self.altitude = 100.0
        self.battery = 100.0
        self.timer = self.create_timer(1.0, self.publish_drone_data)
    def publish_drone_data(self):
        msg = DroneData()
        msg.altitude = self.altitude
        msg.battery = self.battery
        self.publisher_.publish(msg)


self.get_logger().info(
```

```
            f'Altitude: {self.altitude:.2f} m | '
            f'Battery: {self.battery:.1f}%'
        )

        self.altitude += 0.5
        self.battery = max(0.0, self.battery - 1.0)


def main(args=None):
    rclpy.init(args=args)
    node = DronePublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

Note: max(0.0, ...) keeps the simulated battery at 0% instead of allowing it to become negative. If documenting the exact file used in your successful run, use the version from your workspace.

7.6 Updated subscriber — altitude and battery

```
import rclpy
from rclpy.node import Node
from drone_interfaces.msg import DroneData
class DroneSubscriber(Node):
    def __init__(self):
        super().__init__('drone_subscriber')
        self.subscription = self.create_subscription(
```

```
            DroneData,
            '/drone_data',
            self.drone_data_callback,
            10
        )
    def drone_data_callback(self, msg):
        self.get_logger().info(
            f'Altitude: {msg.altitude:.2f} m | '
            f'Battery: {msg.battery:.1f}%'
        )
def main(args=None):
    rclpy.init(args=args)
    node = DroneSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```

7.7 Update setup.py

```
entry_points={
    'console_scripts': [
        'drone_publisher = drone_telemetry.drone_publisher:main',
        'drone_subscriber = drone_telemetry.drone_subscriber:main',
    ],
},
```

8. Build and Run the Packages

8.1 Build

From the workspace root, build the interface package and its dependent telemetry package. The command below builds the dependencies required by drone_telemetry:

```
cd ~/ros2_ws
colcon build --packages-up-to drone_telemetry
```

Wait for the build to finish. If an error appears, read the first meaningful error message and correct the package or dependency configuration before running the nodes.

8.2 Source the workspace

```
source install/setup.bash
```

Run this command in each new terminal before using the workspace's ROS 2 executables or custom message. If the ConstructSim terminal already sources the workspace automatically, sourcing again is normally fine.

8.3 Run the publisher

```
ros2 run drone_telemetry drone_publisher
```

Expected output should repeatedly show values similar to:

```
Altitude: 100.00 m | Battery: 100.0%
Altitude: 100.50 m | Battery: 99.0%
Altitude: 101.00 m | Battery: 98.0%
```

8.4 Run the subscriber in another terminal

```
cd ~/ros2_ws
source install/setup.bash
ros2 run drone_telemetry drone_subscriber
```

The subscriber should display the altitude and battery values it receives. Start the publisher and subscriber in separate terminals; do not close the publisher terminal while testing.

```
user:~$ cd ~/ros2_ws
user:~/ros2_ws$ source install/setup.bash
user:~/ros2_ws$ ros2 run drone_telemetry drone_subscriber
[INFO] [1791592838.875835535] [drone_subscriber]: Altitude: 164.00 m | Battery: -28.0%
[INFO] [1791592839.864977037] [drone_subscriber]: Altitude: 164.50 m | Battery: -29.0%
[INFO] [1791592840.863626286] [drone_subscriber]: Altitude: 165.00 m | Battery: -30.0%
[INFO] [1791592841.863762063] [drone_subscriber]: Altitude: 165.50 m | Battery: -31.0%
[INFO] [1791592842.863662245] [drone_subscriber]: Altitude: 166.00 m | Battery: -32.0%
[INFO] [1791592843.863770604] [drone_subscriber]: Altitude: 166.50 m | Battery: -33.0%
[INFO] [1791592844.863433561] [drone_subscriber]: Altitude: 167.00 m | Battery: -34.0%
[INFO] [1791592845.863908666] [drone_subscriber]: Altitude: 167.50 m | Battery: -35.0%
[INFO] [1791592846.863726630] [drone_subscriber]: Altitude: 168.00 m | Battery: -36.0%
[INFO] [1791592847.864143029] [drone_subscriber]: Altitude: 168.50 m | Battery: -37.0%
[INFO] [1791592848.864117589] [drone_subscriber]: Altitude: 169.00 m | Battery: -38.0%
[INFO] [1791592849.863683889] [drone_subscriber]: Altitude: 169.50 m | Battery: -39.0%
[INFO] [1791592850.865199308] [drone_subscriber]: Altitude: 170.00 m | Battery: -40.0%
[INFO] [1791592851.864257316] [drone_subscriber]: Altitude: 170.50 m | Battery: -41.0%
[INFO] [1791592852.863945554] [drone_subscriber]: Altitude: 171.00 m | Battery: -42.0%
[INFO] [1791592853.864650776] [drone_subscriber]: Altitude: 171.50 m | Battery: -43.0%
[INFO] [1791592854.863862951] [drone_subscriber]: Altitude: 172.00 m | Battery: -44.0%
[INFO] [1791592855.863870888] [drone_subscriber]: Altitude: 172.50 m | Battery: -45.0%
[INFO] [1791592856.863563441] [drone_subscriber]: Altitude: 173.00 m | Battery: -46.0%
[INFO] [1791592857.863871336] [drone_subscriber]: Altitude: 173.50 m | Battery: -47.0%
[INFO] [1791592858.863709749] [drone_subscriber]: Altitude: 174.00 m | Battery: -48.0%
[INFO] [1791592859.863648849] [drone_subscriber]: Altitude: 174.50 m | Battery: -49.0%
[INFO] [1791592860.863669497] [drone_subscriber]: Altitude: 175.00 m | Battery: -50.0%
[INFO] [1791592861.888145196] [drone_subscriber]: Altitude: 175.50 m | Battery: -51.0%
[INFO] [1791592862.863698902] [drone_subscriber]: Altitude: 176.00 m | Battery: -52.0%
[INFO] [1791592863.863738670] [drone_subscriber]: Altitude: 176.50 m | Battery: -53.0%
[INFO] [1791592864.864210573] [drone_subscriber]: Altitude: 177.00 m | Battery: -54.0%
[INFO] [1791592865.863951456] [drone_subscriber]: Altitude: 177.50 m | Battery: -55.0%
[INFO] [1791592866.863784103] [drone_subscriber]: Altitude: 178.00 m | Battery: -56.0%
```

Figure 6: Subscriber Node With Both Altitude & Battery Level

9. Verification and Troubleshooting

9.1 Useful verification commands

List active ROS 2 nodes

ros2 node list

List available topics

ros2 topic list

Inspect the combined telemetry topic

ros2 topic info /drone_data

```
# Print messages directly from the topic
ros2 topic echo /drone_data

# Inspect the custom message definition
ros2 interface show drone_interfaces/msg/DroneData
```

10. Results and Conclusion

Results

- Created a ROS 2 publisher node for simulated drone altitude.
- Created an optional subscriber node to display received altitude values in real time.
- Defined a custom DroneData message with altitude and battery fields.
- Updated the publisher and subscriber to exchange both telemetry values over /drone_data.
- Built and tested the nodes in ConstructSim using ROS 2 Humble.

Conclusion

This practical demonstrated the basic ROS 2 publish–subscribe communication pattern. A publisher node generated simulated drone telemetry, and a subscriber received and displayed the data. The custom DroneData message extended the application so altitude and battery percentage could be transferred together. The exercise provided practical experience with nodes, topics, messages, callbacks, packages, workspace builds and basic debugging commands.

Report By:

Sujana (U25UV24T043102)

5TH Sem ECE

Batch -7 Aviation Student